HDOC Day-70

Digital Root Using Recursion and a Static Call Counter

Problem Statement

Write a recursive C program to calculate the digital root of a non-negative integer. Use a static variable to count calls to the digital_root() recursive function. Repeatedly sum the digits until a single digit remains.

Algorithm

1. Start and read a non-negative integer n.

2. Call digital_root(n).

3. In digital_root(), increment a static call counter on every invocation.

4. If n is a single digit (0 to 9), display it as the digital root and display the recursive call counter; return.

5. Otherwise, use the recursive helper digit_sum(n) to add all digits of n.

6. Recursively call digital_root() with the computed sum.

7. Stop after the final single-digit result is displayed.

Counting convention: the displayed counter counts calls to digital_root(), including the initial call. Calls to the helper digit_sum() are not included.

Test Case Table

Test case	Input	Digital root	digital_root() calls
1	9875	2	4
2	0	0	1
3	7	7	1
4	29	2	3
5	999	9	3
6	12345	6	3
7	9999	9	3
8	10	1	2
9	38	2	3
10	123456789	9	3

C Program

```
#include <stdio.h>

unsigned int digit_sum(unsigned long long n)
{
    if (n < 10)
        return (unsigned int)n;
    return (unsigned int)(n % 10) + digit_sum(n / 10);
}
```

```
unsigned int digital_root(unsigned long long n)
{
    static unsigned int calls = 0;
    calls++;

    if (n < 10)
    {
        printf("Digital root: %llu\n", n);
        printf("Recursive calls to digital_root: %u\n", calls);
        return (unsigned int)n;
    }

    return digital_root(digit_sum(n));
}

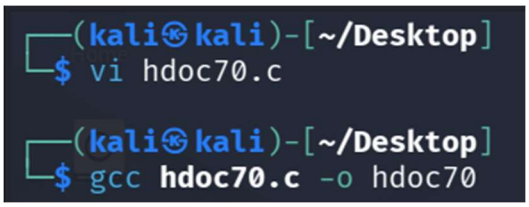
int main(void)
{
    unsigned long long number;
    printf("Enter a non-negative integer: ");
    if (scanf("%llu", &number) != 1)
    {
        printf("Invalid input.\n");
        return 1;
    }
    digital_root(number);
    return 0;
}
```

Compilation and Execution

Compile: gcc hdoc70.c -o hdoc70

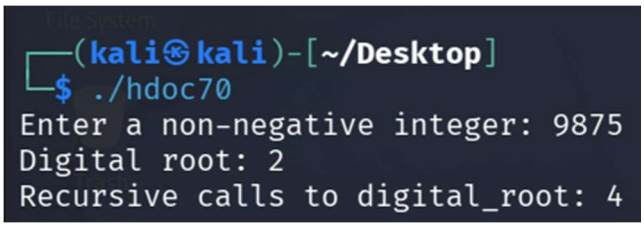
Execute: ./hdoc70

Compilation and Execution - Screenshot Space



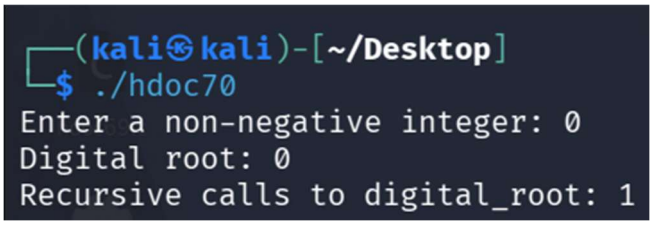
Test Case 1 - Screenshot Space

Enter: 9875 | Expected digital root: 2 | Expected recursive calls: 4. Insert the terminal output screenshot below.



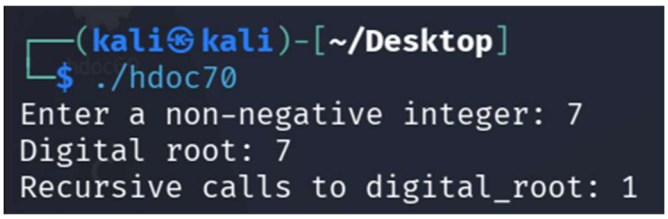
Test Case 2 - Screenshot Space

Enter: 0 | Expected digital root: 0 | Expected recursive calls: 1. Insert the terminal output screenshot below.



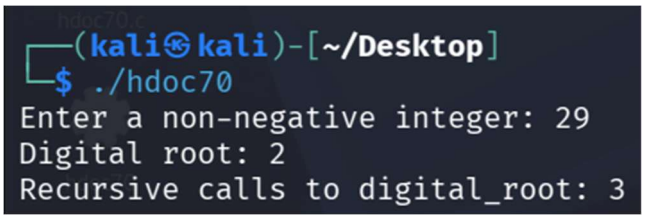
Test Case 3 - Screenshot Space

Enter: 7 | Expected digital root: 7 | Expected recursive calls: 1. Insert the terminal output screenshot below.



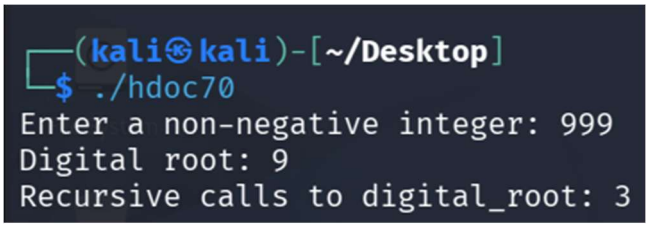
Test Case 4 - Screenshot Space

Enter: 29 | Expected digital root: 2 | Expected recursive calls: 3. Insert the terminal output screenshot below.



Test Case 5 - Screenshot Space

Enter: 999 | Expected digital root: 9 | Expected recursive calls: 3. Insert the terminal output screenshot below.



Test Case 6 - Screenshot Space

Enter: 12345 | Expected digital root: 6 | Expected recursive calls: 3. Insert the terminal output screenshot below.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc70
Enter a non-negative integer: 12345
Digital root: 6
Recursive calls to digital_root: 3
```

Test Case 7 - Screenshot Space

Enter: 9999 | Expected digital root: 9 | Expected recursive calls: 3. Insert the terminal output screenshot below.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc70
Enter a non-negative integer: 9999
Digital root: 9
Recursive calls to digital_root: 3
```

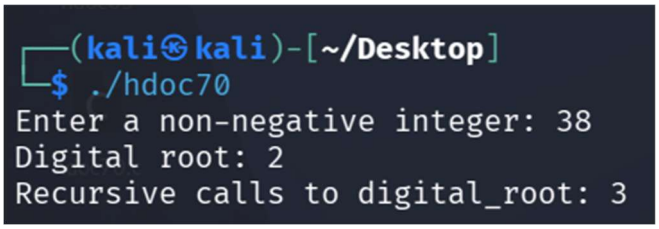
Test Case 8 - Screenshot Space

Enter: 10 | Expected digital root: 1 | Expected recursive calls: 2. Insert the terminal output screenshot below.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc70
Enter a non-negative integer: 10
Digital root: 1
Recursive calls to digital_root: 2
```

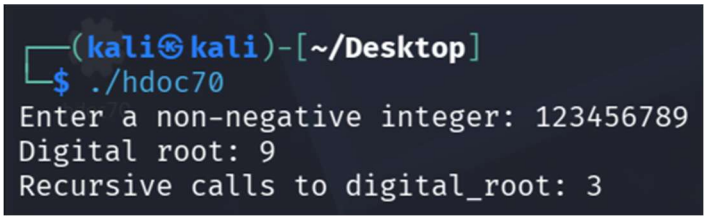
Test Case 9 - Screenshot Space

Enter: 38 | Expected digital root: 2 | Expected recursive calls: 3. Insert the terminal output screenshot below.



Test Case 10 - Screenshot Space

Enter: 123456789 | Expected digital root: 9 | Expected recursive calls: 3. Insert the terminal output screenshot below.



Result

The program calculates the digital root recursively and reports the number of calls to digital_root() using a static variable.